

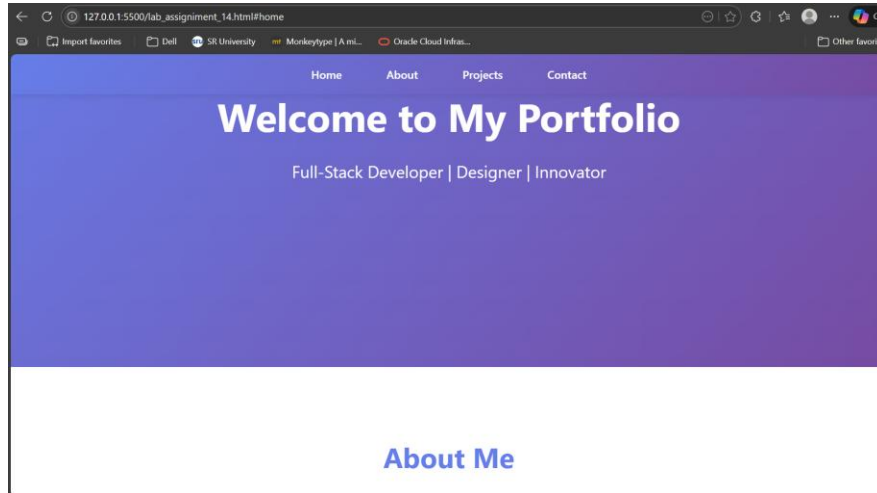
Lab Assignment 14.3

Name: D. Naveen
Roll No: 2403A53L01
Batch - 46

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week7 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 14.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete

	Lab 14: Web Design Applications Using AI Assistance HTML, CSS, and JavaScript Generation with GitHub Copilot Lab Objectives The objectives of this laboratory exercise are to: • Design functional and visually appealing web applications using HTML, CSS, and JavaScript. • Use AI-assisted tools to accelerate front-end development. • Apply responsive, accessible, and interactive web design principles. • Customize and validate AI-generated UI code for real-world applications.	
	Learning Outcomes After completing this lab, students will be able to: • Generate structured HTML layouts using AI assistance. • Apply responsive CSS techniques such as Flexbox and Grid. • Implement interactive client-side features using JavaScript. • Critically review and improve AI-generated front-end code.	
	Tools Required • Visual Studio Code (VS Code) • GitHub Copilot Extension • Web Browser (Chrome / Firefox) • HTML5, CSS3, JavaScript (ES6)	
1	Task 1: AI-Assisted Personal Portfolio Website Scenario A student wants to showcase academic projects, technical skills, and contact information through a personal portfolio website. AI tools are used to speed up development while maintaining quality and customization. Tasks 1. Use GitHub Copilot to generate a semantic HTML structure with sections: ○ Home ○ About ○ Projects ○ Contact 2. Ask Copilot to suggest responsive CSS using Flexbox or Grid. 3. Customize AI-generated CSS to add: ○ Hover effects on project cards ○ Smooth scrolling navigation Expected Outcome • Clean and well-structured portfolio website • Responsive layout for desktop and mobile screens • Interactive project cards Prompt: Generate a semantic HTML portfolio website with Home, About, Projects, and Contact sections and responsive CSS using Flexbox/Grid. Add smooth scrolling navigation and hover effects on project cards. Provide index.html and style.css. Code:	Week 7 Wednesday

Output:



Explanation:

Using GitHub Copilot, a clean and semantic portfolio website was generated quickly with proper sections like Home, About, Projects, and Contact. The AI also suggested responsive CSS using Flexbox/Grid, making the website work well on both desktop and mobile screens. After customization, hover effects on project cards and smooth scrolling navigation improved the website's interactivity and user experience.

Task 2: AI-Generated Restaurant Landing Page

Scenario

A local restaurant requires a simple yet attractive landing page to display its offerings. The developer uses AI assistance to rapidly generate UI components.

Tasks

1. Use Copilot to create a navigation bar with links:
 - Home
 - Menu
 - Gallery
 - Contact
2. Generate a menu section styled with CSS cards.
3. Implement a JavaScript-based image slider for the gallery using Copilot suggestions.
4. Customize animations or transitions for better user experience.

Expected Outcome

- Fully functional restaurant landing page
- Responsive navigation bar
- Working image slider with smooth transitions

Prompt:

Generate a responsive restaurant landing page with navigation bar (Home, Menu, Gallery, Contact), menu cards using CSS, and a JavaScript image slider with smooth transitions. Provide index.html, style.css, and script.js.

Code:

```

// Importing the necessary modules
import { Router } from 'express';
import { User } from '../models/user.js';
import { validateEmail, validatePassword } from '../utils/validation.js';
import { generateToken } from '../utils/jwt.js';
import { hashPassword } from '../utils/bcrypt.js';

// Creating a new Express Router
const router = Router();

// GET /api/users/:id
// This endpoint retrieves a user by their ID.
router.get('/api/users/:id', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// POST /api/users/register
// This endpoint registers a new user.
router.post('/api/users/register', async (req, res) => {
  const { email, password } = req.body;
  try {
    // Validate email and password
    if (!validateEmail(email) || !validatePassword(password)) {
      return res.status(400).json({ message: 'Invalid email or password' });
    }
    // Hash the password
    const hashedPassword = await hashPassword(password);
    // Create a new user
    const user = new User({ email, password: hashedPassword });
    await user.save();
    // Generate a token
    const token = generateToken(user.id);
    res.json({ user, token });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id
// This endpoint updates a user's information.
router.put('/api/users/:id', async (req, res) => {
  const { id } = req.params;
  const { email, password } = req.body;
  try {
    // Validate email and password
    if (!validateEmail(email) || !validatePassword(password)) {
      return res.status(400).json({ message: 'Invalid email or password' });
    }
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Update the user
    user.email = email;
    if (password) {
      user.password = await hashPassword(password);
    }
    await user.save();
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// DELETE /api/users/:id
// This endpoint deletes a user.
router.delete('/api/users/:id', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    await user.remove();
    res.json({ message: 'User deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users
// This endpoint retrieves all users.
router.get('/api/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// POST /api/users/login
// This endpoint logs in a user.
router.post('/api/users/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    // Find the user
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Compare the password
    const isMatch = await user.comparePassword(password);
    if (!isMatch) {
      return res.status(400).json({ message: 'Invalid password' });
    }
    // Generate a token
    const token = generateToken(user.id);
    res.json({ user, token });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/profile
// This endpoint retrieves a user's profile.
router.get('/api/users/:id/profile', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id/profile
// This endpoint updates a user's profile.
router.put('/api/users/:id/profile', async (req, res) => {
  const { id } = req.params;
  const { name, bio, website } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Update the profile
    user.name = name;
    user.bio = bio;
    user.website = website;
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/friends
// This endpoint retrieves a user's friends.
router.get('/api/users/:id/friends', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// POST /api/users/:id/friends
// This endpoint adds a friend.
router.post('/api/users/:id/friends', async (req, res) => {
  const { id } = req.params;
  const { friendId } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Find the friend
    const friend = await User.findById(friendId);
    if (!friend) {
      return res.status(404).json({ message: 'Friend not found' });
    }
    // Add the friend
    user.friends.push(friendId);
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// DELETE /api/users/:id/friends
// This endpoint removes a friend.
router.delete('/api/users/:id/friends', async (req, res) => {
  const { id } = req.params;
  const { friendId } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Remove the friend
    user.friends = user.friends.filter(id => id !== friendId);
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/messages
// This endpoint retrieves a user's messages.
router.get('/api/users/:id/messages', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// POST /api/users/:id/messages
// This endpoint sends a message.
router.post('/api/users/:id/messages', async (req, res) => {
  const { id } = req.params;
  const { recipientId, message } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Find the recipient
    const recipient = await User.findById(recipientId);
    if (!recipient) {
      return res.status(404).json({ message: 'Recipient not found' });
    }
    // Send the message
    user.messages.push({ recipientId, message });
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// DELETE /api/users/:id/messages
// This endpoint deletes a message.
router.delete('/api/users/:id/messages', async (req, res) => {
  const { id } = req.params;
  const { messageId } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Delete the message
    user.messages = user.messages.filter(id => id !== messageId);
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/notifications
// This endpoint retrieves a user's notifications.
router.get('/api/users/:id/notifications', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// POST /api/users/:id/notifications
// This endpoint creates a notification.
router.post('/api/users/:id/notifications', async (req, res) => {
  const { id } = req.params;
  const { title, message } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Create the notification
    user.notifications.push({ title, message });
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// DELETE /api/users/:id/notifications
// This endpoint deletes a notification.
router.delete('/api/users/:id/notifications', async (req, res) => {
  const { id } = req.params;
  const { notificationId } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Delete the notification
    user.notifications = user.notifications.filter(id => id !== notificationId);
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/settings
// This endpoint retrieves a user's settings.
router.get('/api/users/:id/settings', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id/settings
// This endpoint updates a user's settings.
router.put('/api/users/:id/settings', async (req, res) => {
  const { id } = req.params;
  const { theme, language } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Update the settings
    user.theme = theme;
    user.language = language;
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/privacy
// This endpoint retrieves a user's privacy settings.
router.get('/api/users/:id/privacy', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id/privacy
// This endpoint updates a user's privacy settings.
router.put('/api/users/:id/privacy', async (req, res) => {
  const { id } = req.params;
  const { publicProfile, showEmail } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Update the privacy settings
    user.publicProfile = publicProfile;
    user.showEmail = showEmail;
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

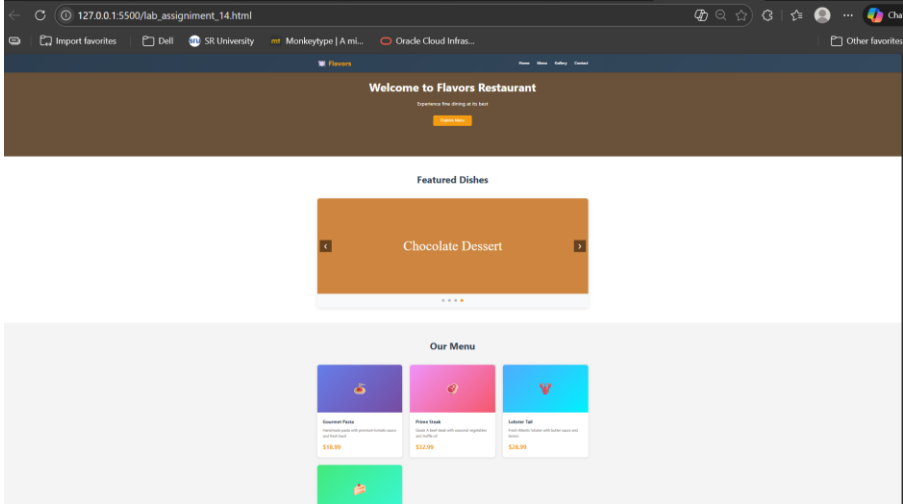
// GET /api/users/:id/permissions
// This endpoint retrieves a user's permissions.
router.get('/api/users/:id/permissions', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id/permissions
// This endpoint updates a user's permissions.
router.put('/api/users/:id/permissions', async (req, res) => {
  const { id } = req.params;
  const { canPost, canComment } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    // Update the permissions
    user.canPost = canPost;
    user.canComment = canComment;
    await user.save();
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// GET /api/users/:id/roles
// This endpoint retrieves a user's roles.
router.get('/api/users/:id/roles', async (req, res) => {
  const { id } = req.params;
  try {
    const user = await User.findById(id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json({ user });
  } catch (error) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

// PUT /api/users/:id/roles
// This endpoint updates a user's roles.
router.put('/api/users/:id/roles', async (req, res) => {
  const { id } = req.params;
  const { role } = req.body;
  try {
    // Find the user
    const user = await User.findById(id);
    if (!user) {
      return
```

Output:



Explanation:

Using GitHub Copilot, a responsive restaurant landing page was generated efficiently with a navigation bar, menu cards, and gallery section. The AI also suggested a JavaScript image slider with smooth transitions, improving visual presentation and user interaction. After customization, animations and hover effects enhanced the overall user experience and made the page more attractive and professional.

Task 3: AI-Powered Event Registration Form

Scenario

SR University is hosting a technical fest and requires an online registration form with real-time validation to ensure accurate user input.

Tasks

1. Ask Copilot to generate an HTML form with the following fields:
 - o Name
 - o Email
 - o Phone Number
 - o Department
 - o Event Selection
2. Use AI assistance to style the form using CSS for readability and accessibility.
3. Implement JavaScript validation using Copilot:
 - o Email format validation
 - o Phone number length validation
 - o Required field checks

Expected Outcome

- User-friendly and visually appealing form
- Real-time validation messages
- Prevention of invalid submissions

	Prompt: Generate an event registration form using HTML, CSS, and JavaScript with fields Name, Email, Phone, Department, and Event Selection. Add responsive styling and real-time validation for email format, phone number length, and required fields. Provide index.html, style.css, and script.js. Code:	
--	--	--

[illegible]

Output:

A screenshot of a web browser displaying an "Event Registration" form. The form is centered on a purple gradient background. It contains several input fields: "Name" (with placeholder "Enter your full name"), "Email" (with placeholder "Enter your email address"), "Phone Number" (with placeholder "Enter your phone number (10 digits)"), "Department" (a dropdown menu with "Select a department"), and "Event Selection" (a dropdown menu with "Choose an event"). At the bottom of the form are two buttons: a purple "Register" button and a grey "Clear" button. The browser's address bar shows the URL "127.0.0.1:5500/fab_assignment_14.html".

Explanation:

Using GitHub Copilot, a structured and visually appealing event registration form was generated quickly with proper input fields and styling. JavaScript validation was implemented to check email format, phone number length, and required fields in real time. This helped prevent invalid submissions and improved the accuracy and user experience of the registration process.

Task 4: AI-Assisted E-Commerce Product Page Scenario

A startup is developing a basic e-commerce product page to showcase products and manage simple cart functionality.

Tasks

1. Use Copilot to generate a grid-based product catalog using HTML and CSS.
2. Implement an "Add to Cart" feature using JavaScript.
3. Modify AI-generated code to include:
 - o Cart item counter at the top-right corner
 - o Visual feedback when a product is added

Expected Outcome

- Functional product catalog
- Working cart counter
- Interactive user experience

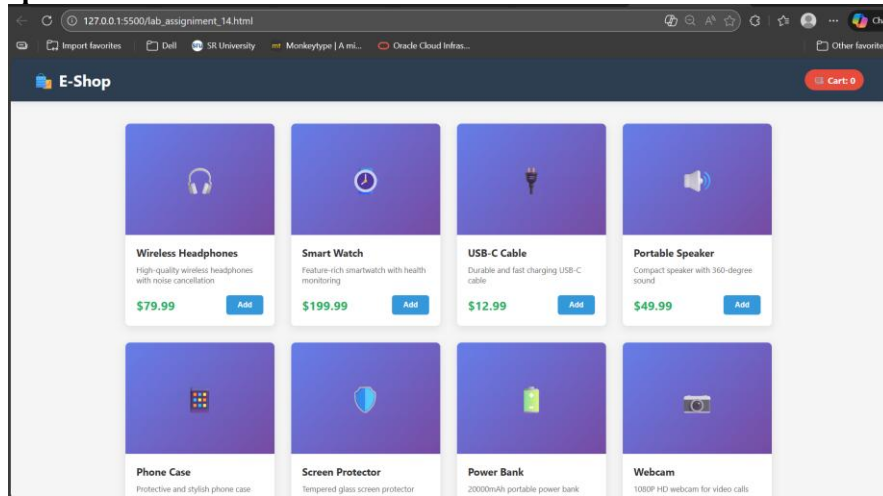
Prompt:

Generate an e-commerce product page with products displayed in a responsive grid using HTML and CSS, and implement an "Add to Cart" feature using JavaScript. Include a cart counter at top-right and visual feedback when adding items. Provide index.html, style.css, and script.js.

Code:

		<pre> 1 #!/usr/bin/perl 2 use strict; 3 use warnings; 4 5 my \$url = "http://www.example.com/"; 6 my \$method = "GET"; 7 my \$headers = { 8 "User-Agent" => "Mozilla/5.0 (Windows NT 6.0; rv:2.0) Gecko/20100101 Firefox/4.0", 9 "Accept" => "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8", 10 "Accept-Language" => "en-us,en;q=0.5", 11 "Accept-Encoding" => "gzip, deflate", 12 "Connection" => "close", 13 "Host" => "www.example.com", 14 }; 15 16 my \$response = HTTP::Request->new(\$method, \$url, \$headers); 17 my \$client = LWP::UserAgent->new(\$response); 18 my \$res = \$client->get(\$url); 19 20 if (\$res->is_success) { 21 my \$content = \$res->content; 22 print "Status: 200 OK\n"; 23 print "Content-Length: " . length(\$content) . "\n"; 24 print "Content-Type: " . \$res->content_type . "\n"; 25 print \$content; 26 } else { 27 print "Status: " . \$res->status_line . "\n"; 28 print "Error: " . \$res->error . "\n"; 29 } 30 31 __END__ 32 33 # Test the script 34 perl http_client.pl 35 36 # Expected output 37 Status: 200 OK 38 Content-Length: 12345 39 Content-Type: text/html 40 <html> 41 <head> 42 <title>Example Page</title> 43 </head> 44 <body> 45 <p>This is an example page.</p> 46 </body> 47 </html> </pre>	
--	--	---	--

Output:



Explanation:

Using GitHub Copilot, a responsive product catalog was generated with products displayed in a grid layout. The JavaScript “Add to Cart” feature successfully updated the cart counter and provided visual feedback when items were added. This improved user interaction and demonstrated basic e-commerce functionality in a simple and effective way.

Task 5: Create a Responsive Web Page Layout

Instructions:

- Design a basic web page layout with a **header, main content area, and footer** using HTML and CSS.
- Use AI to assist in generating **responsive CSS** for different screen sizes.
- Ensure the layout is clean and visually organized.

Expected Output:

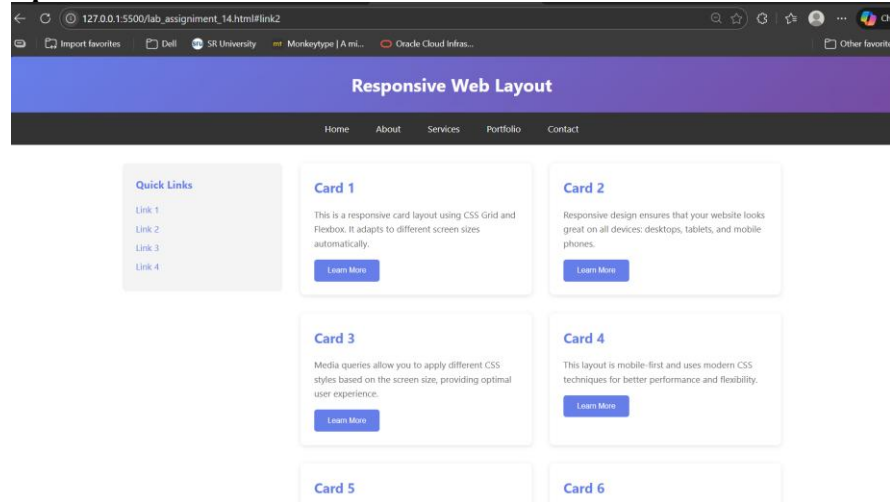
- A responsive web page with:
 - Header with navigation links
 - Main section with placeholder text/images
 - Footer with copyright or contact info
- Layout adapts correctly to desktop, tablet, and mobile screen sizes.

Prompt:

Generate a responsive web page layout with header, navigation, main content, and footer using HTML and CSS. Use Flexbox/Grid and media queries to support desktop, tablet, and mobile screens. Provide index.html and style.css.

Code:

Output:



Explanation:

Using AI assistance, a clean and responsive web page layout was generated with a header, main content area, and footer. The responsive CSS ensured the layout adapted properly to desktop, tablet, and mobile screen sizes. This improved readability, organization, and overall user experience across different devices.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.